

Data Types, Constraints & SQL Statements

Comprehensive Reference for Schema Design, Column Types & SQL Syntax

1. 1234 Numeric Data Types

Numeric data types define columns that store numerical values for quantitative calculations, measurements, and identifiers.

TYPE	DESCRIPTION / USED FOR	EXAMPLE VALUES
Integer (INT)	Stores whole numbers (positive, negative, or zero). No fractions.	5 , 100 , -250
Decimal / NUMERIC	Stores exact numbers with fractional parts. Best for exact currency.	80.90 , 12.50
TINYINT	Stores very small integer values (1 byte: -128 to 127, or 0 to 255 unsigned).	1 , 127 , 255

Key Numeric Concepts

- Integer vs. Decimal:** INT strictly holds whole numbers. DECIMAL(precision, scale) retains fractional precision. Placing 80 into a DECIMAL stores it as 80.00 .
- UNSIGNED Attribute:** Disallows negative numbers, doubling the positive integer capacity range.

2. String Data Types

String data types store textual data, alphanumeric combinations, and special characters.

TYPE	LENGTH BEHAVIOR	STORAGE & BEST USAGE
CHAR(N)	Fixed Length	Pads unused space. Ideal for uniform fixed-size keys (e.g., country codes, hashes).
VARCHAR(N)	Variable Length	Dynamically adjusts size based on input up to limit N. Ideal for names, emails, and titles.
TINYTEXT	Variable Length	Up to ~255 bytes / characters. Small text notes.
TEXT	Variable Length	Up to ~65,535 bytes (~65 KB). Standard articles and posts.
MEDIUMTEXT	Variable Length	Up to ~16.7 million bytes (~16 MB). Large documents or book chapters.
LONGTEXT	Variable Length	Up to ~4 GB of data. Massive logs or text archives.


CHAR vs VARCHAR: CHAR = Fixed Length  | VARCHAR = Variable Length 

3. Database Constraints

Constraints enforce business rules and data integrity, preventing invalid data from entering table schemas.

Core Constraints

- **NOT NULL:** Ensures a column cannot contain missing or `NULL` values. Value entry is mandatory.
- **DEFAULT:** Provides a predefined automatic value if no value is explicitly supplied during insertion.
- **Constraint Scope:** Applied at the *Column Level* (single column rule) or *Table Level* (multi-column rules or foreign keys).

4. Complete SQL Command Examples

A. Database Management (CREATE / DROP)

```
-- Create a new database
CREATE DATABASE company_db;

-- Remove a database permanently
DROP DATABASE company_db;
```

B. Table Creation (CREATE TABLE with Constraints)

```
-- Create customer table with constraints
CREATE TABLE customer (
    customer_id INT NOT NULL,
    customer_name VARCHAR(50) NOT NULL,
    email VARCHAR(100),
    PRIMARY KEY (customer_id)
);

-- Create player table with DEFAULT constraint
CREATE TABLE player (
    player_id INT PRIMARY KEY,
    player_name VARCHAR(50) NOT NULL,
    city VARCHAR(50) DEFAULT 'Barcelona'
);
```

C. Table Alteration (ALTER, MODIFY, DROP)

```
-- Add a new column to an existing table
ALTER TABLE customer ADD COLUMN phone VARCHAR(20);

-- Modify column data type or constraints
ALTER TABLE customer MODIFY COLUMN customer_name VARCHAR(100) NOT NULL;

-- Drop a column from a table
ALTER TABLE customer DROP COLUMN phone;

-- Drop entire table permanently
DROP TABLE player;
```

D. Data Insertion (INSERT ONE / INSERT MANY)

```
-- Insert a single record
INSERT INTO customer (customer_id, customer_name, email)
VALUES (1, 'Mark Simpson', 'mark@example.com');

-- Insert relying on DEFAULT value for city ('Barcelona')
INSERT INTO player (player_id, player_name)
VALUES (101, 'Mark');

-- Insert multiple records in a single query
INSERT INTO customer (customer_id, customer_name, email)
VALUES
  (2, 'Sara Ahmed', 'sara@example.com'),
  (3, 'Ali Hassan', 'ali@example.com'),
  (4, 'John Doe', 'john@example.com');
```

E. Querying Data (SELECT Statements)

```
-- Select all columns from a table
SELECT * FROM customer;

-- Select specific columns with filtered conditions
SELECT customer_name, email
FROM customer
WHERE customer_id = 1;
```

F. Copying Data Across Tables (INSERT INTO SELECT)

```
-- Copy matching records from one table into another existing table
INSERT INTO VIP_Customers (customer_id, customer_name)
SELECT customer_id, customer_name
FROM customer
WHERE customer_id <= 2;
```

G. Updating & Deleting Data (UPDATE / DELETE)

```
-- Update existing row data
UPDATE customer
SET email = 'mark.simpson@newdomain.com'
WHERE customer_id = 1;

-- Delete specific records based on condition
DELETE FROM customer
WHERE customer_id = 4;

-- Delete all rows in a table (keep table structure)
DELETE FROM customer;
```